

HLD Document

High-Level Design

System Architecture & Overview

Project	RAG-Based Customer Support Assistant
Technology	LangGraph + ChromaDB + Groq LLM + HuggingFace
Document Type	HLD Document
Date	April 2025

1. System Overview

Problem Definition

Traditional customer support systems rely on rigid, rule-based chatbots that cannot handle dynamic, context-sensitive queries. They fail when users ask questions outside predefined scripts, leading to poor customer experience. This project solves that by building a Retrieval-Augmented Generation (RAG) system that reads from a real knowledge base PDF and provides intelligent, contextual answers — while escalating complex queries to a human agent via a Human-in-the-Loop (HITL) mechanism.

Scope of the System

- Accept natural language queries from users via a CLI interface
- Load and process a PDF knowledge base into a vector store
- Retrieve semantically relevant chunks using embedding similarity
- Generate accurate answers using a free LLM (Groq/LLaMA)
- Route queries through a LangGraph-based workflow
- Escalate low-confidence or complex queries to human agents (HITL)

2. Architecture Diagram

The following diagram illustrates the complete system architecture including document ingestion, query processing, LangGraph workflow orchestration, and Human-in-the-Loop escalation:

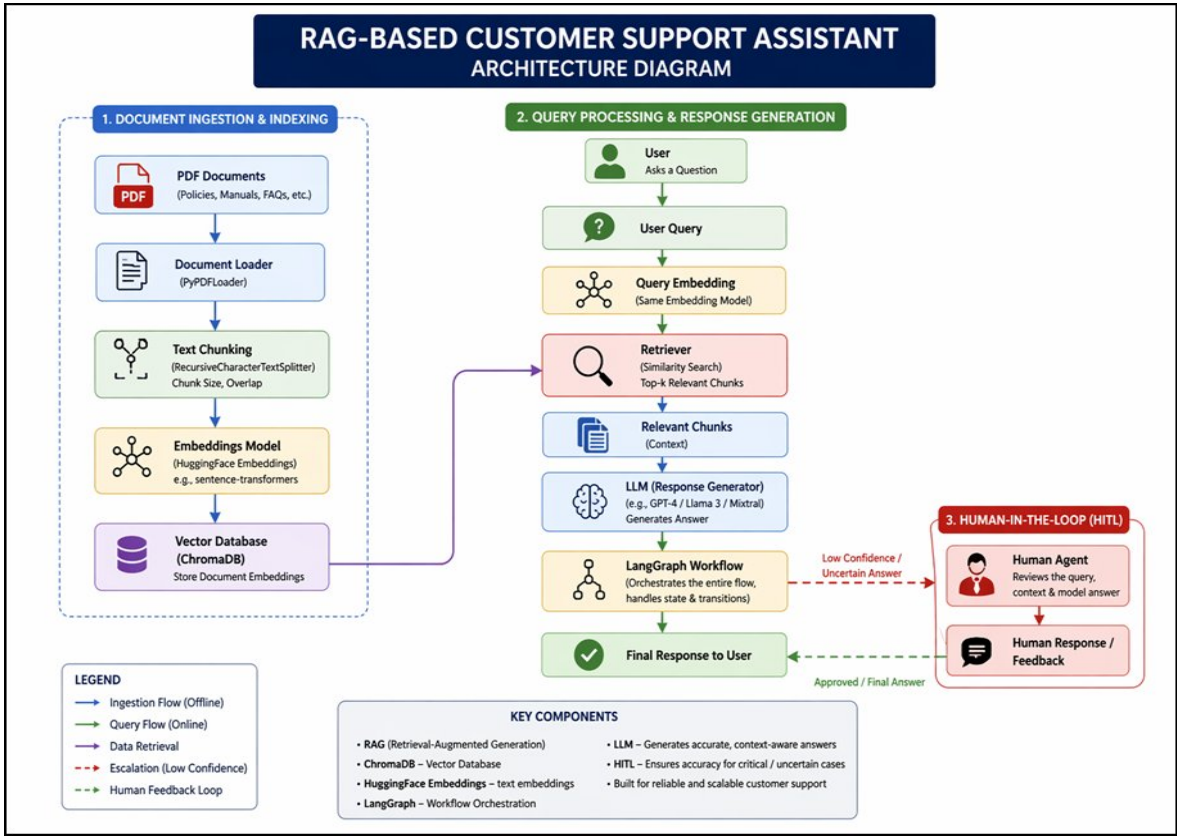


Figure 1: RAG-Based Customer Support Assistant — System Architecture Diagram

3. Data Flow

Document Ingestion Pipeline (runs once at startup):

Step 1	PDF Loading	PyPDFLoader reads knowledge_base.pdf into Document objects
Step 2	Chunking	RecursiveCharacterTextSplitter splits into 500-char chunks with 50-char overlap
Step 3	Embedding	HuggingFace model converts each chunk into a 384-dim vector
Step 4	Storage	Chroma stores vectors + metadata in ./chroma_db on disk

Query Lifecycle (runs per user query):

Step 1	User Input	User types a natural language question in CLI
Step 2	Retrieve Node	LangGraph invokes retriever → fetches top-3 chunks from ChromaDB
Step 3	Confidence Check	Router checks context length and query keywords
Step 4a	Generate Node	If confident → LLM generates answer from context + query
Step 4b	Escalate Node	If low confidence → HITL message displayed, human notified
Step 5	Output	Answer printed to CLI; loop continues

4. Technology Choices & Justification

Technology	Choice	Justification
Vector DB	ChromaDB	Lightweight, local, no server needed, easy Python integration, persistent storage
Workflow	LangGraph	Graph-based state machine, supports conditional routing, HITL, production-ready
LLM	Groq LLaMA-3.1-8b	Free API, fast inference (low latency), no cost for internship project
Embeddings	HuggingFace all-MiniLM-L6-v2	Free, runs locally, 384-dim vectors, excellent semantic accuracy
PDF Loader	PyPDFLoader (LangChain)	Standard, handles multi-page PDFs, integrates with LangChain pipeline
Language	Python 3.10+	Industry standard for AI/ML, rich ecosystem, LangChain support

5. Scalability Considerations

Large Documents: ChromaDB supports millions of vectors. Chunk size (500) ensures memory efficiency. For very large corpora, switch to Pinecone or Weaviate managed vector DBs.

High Query Load: Groq API supports concurrent requests. For higher load, implement request queuing and response caching for repeated queries.

Latency: HuggingFace embeddings run locally (no API call). Groq LLM has sub-second response times. ChromaDB local lookup is near-instant.

Multi-Document: Extend PyPDFLoader to DirectoryLoader for bulk ingestion. Use metadata filtering in ChromaDB to scope retrieval per document.